

Smart Hospital System (C Language)

CSC 1012 Introduction to Computer Programming

University Of Sri Jayewardenepura



By:

Name: S.P Gunasekara

Index: AS20250616

Submitted Date : 20/09/2026

Content

- 1. Introduction and Objectives**
- 2. High-Level Architecture**
- 3. Data Structure Design**
- 4. List of Implemented Functions**
- 5. Sample Output**
- 6. Testing**
- 7. GitHub Repository and Assumptions**
- 8. Challenges Faced and Future Improvements**

1. Introduction and Objectives

This project implements a modular, menu-driven Smart Hospital Patient & Resource Allocation System in C, developed for CSC 1012 (Introduction to Computer Programming) at the University of Sri Jayewardenepura. The system simulates core hospital operations: patient intake, bed allocation across four wards, emergency triage prioritisation, and automated medical billing.

Rather than using C structs, the system was built entirely with parallel arrays, as encouraged by the assignment brief, giving hands-on practice with array indexing, custom function design, string handling, loops, conditional logic, mathematical modelling of billing rules, and file-based persistence. The goal was a working system that mirrors a real (simplified) hospital front-desk workflow: a patient is registered, an itemised bill and estimated wait time are generated immediately, and hospital administrators can later pull up occupancy and revenue reports.

2. High-Level Architecture

The program is split into eight source files, each responsible for one part of the system, all declared through a single shared header (`hospital.h`) and coordinated by a menu loop in `main.c`.

lookup.c (Requirement 1) holds the two lookup tables given in the spec — doctor specialties and hospital wards — as constant parallel arrays, plus the `bedOccupancy[4][20]` matrix that tracks which beds are free or occupied. Three functions live here: `initBedOccupancy()` resets every bed to free at program start, `printBedStatus()` displays occupancy counts per ward, and `findFreeBed()` scans a ward's row of the matrix for the next open bed.

patient.c (Requirement 2) declares the nine parallel arrays that hold patient data and implements `registerPatient()`, which walks the user through name, age, urgency level, specialty selection, and (conditionally) ward admission, validating every input before moving on.

billing.c (Requirement 3) implements the six billing formulas from the spec as standalone functions — wait time, emergency surcharge, ward stay cost, gross total, age subsidy discount, and final payable amount — so each formula can be tested and reasoned about independently of the UI.

sort.c (Requirement 4) contains `sortByPriority()`, a bubble sort that orders patients by urgency level (Critical first) while preserving registration order for patients at the same urgency level.

display.c (Requirement 5) contains `printBill()`, which formats and prints the itemised admission and bill exactly as shown in the spec's sample output, including a custom currency formatter that inserts comma thousand separators (C's `printf` has no built-in support for this).

report.c (Requirement 6) contains `printSummaryReport()`, which loops over all registered patients to tally urgency-level counts, total revenue, total discounts granted, per-ward bed occupancy percentages, and the highest-paying patient.

fileio.c (Requirement 7, bonus) persists state across runs: `saveBedStatus()/loadBedStatus()` write and read the bed matrix to `beds_status.txt`, and `appendPatientRecord()` logs every patient permanently to `patient_records.txt`.

main.c ties all of the above together in a five-option menu loop: register a patient (which also auto-generates their bill), view bed status, view the priority-sorted patient list, view the summary report, or exit (which saves bed status to file first).

3. Data Structures Design

The assignment explicitly permits parallel arrays instead of structs, so every patient's data is spread across nine arrays, all indexed by the same patient number *i*:

```
char    patientName[100][50];
int     patientAge[100];
int     patientUrgency[100];  (1 = Normal, 2 = Urgent, 3 = Critical)
int     patientSpecialtyID[100]; (1-4)
int     patientAdmitted[100]; (1 = yes, 0 = no)
int     patientWardID[100];  (1-4, 0 if not admitted)
int     patientBedIndex[100]; (0-based bed slot, -1 if not admitted)
int     patientDaysAdmitted[100];
double  patientFinalBill[100];
double  patientDiscount[100];
```

For example, `patientName[3]`, `patientAge[3]`, and `patientUrgency[3]` all describe the same, fourth registered patient. This avoids the need for a struct `Patient`, at the cost of needing to keep every array in sync manually — a trade-off worth mentioning in a report, since it's a genuine design decision with pros (simplicity, matches lecture material) and cons (no single object to pass around, easy to forget updating one array).

The two lookup tables (specialties and wards) are similarly stored as constant parallel arrays populated directly from the specification's tables, indexed 0-3 for the four entries each.

Bed occupancy uses a 2D integer matrix, `bedOccupancy[4][20]`: each row represents a ward, each column a bed slot within that ward, with 0 meaning free and 1 meaning occupied. Since wards have different capacities (20, 10, 10, and 5 beds), only the first *N* columns of each row are ever used, where *N* is that ward's actual capacity from the lookup table — the rest of the row simply stays unused.

4. List of Implemented Functions

Function	File	Purpose
initBedOccupancy(void)	lookup.c	Resets every bed in the matrix to free (0)
printBedStatus(void)	lookup.c	Prints occupied/total beds for each ward
findFreeBed(int wardIndex)	lookup.c	Returns the first free bed slot in a ward, or -1 if full
registerPatient(void)	patient.c	Collects and validates all patient registration input
calcWaitTime(int specialtyIndex)	billing.c	Queue count x avg time per patient for that specialty
calcSurcharge(int urgencyLevel, double baseFee)	billing.c	0% / 20% / 50% of base fee by urgency
calcWardCost(int daysAdmitted, int wardIndex)	billing.c	Days x ward daily rate, 0 if not admitted
calcGrossTotal(double baseFee, double surcharge, double wardCost)	billing.c	Sum of the three components
calcDiscount(int age, double grossTotal)	billing.c	15% of gross if age < 5 or age > 65, else 0
calcFinalPayable(double grossTotal, double discount)	billing.c	Gross total minus discount
sortByPriority(int order[])	sort.c	Fills an index array sorted by urgency (stable)
printBill(...)	display.c	Prints the full itemised bill in the spec's format
printSummaryReport(void)	report.c	Prints the urgency/revenue/occupancy/top-payer analytics
saveBedStatus(void)	fileio.c	Writes the bed matrix to beds status.txt
loadBedStatus(void)	fileio.c	Reads the bed matrix back in on startup
appendPatientRecord(int i, double finalPayable)	fileio.c	Appends one line per patient to patient records.txt
generateBillForPatient(int i)	main.c	Calls all six billing functions for one patient and prints/logs the result
printPriorityList(void)	main.c	Calls sortByPriority() and prints a readable list
main(void)	main.c	Runs the menu loop

5. Sample Output

```

=====
SMART HOSPITAL MENU
=====
1. Register New Patient (auto-generates bill)
2. View Bed Occupancy Status
3. View Patients in Priority Order
4. View Summary Report
5. Exit
Enter choice: 1

----- New Patient Registration -----
Patient Name: N.P Perera
Patient Age: 65
Urgency Level (1 = Normal, 2 = Urgent, 3 = Critical): 3

Available Specialties:
1. General Practice (OPD) (LKR 1500.00)
2. Paediatrics (LKR 2500.00)
3. Cardiology (LKR 4500.00)
4. Neurology (LKR 5000.00)
Select Specialty ID (1-4): 4

Is Patient Admitted to a Ward? (1 = Yes, 0 = No): 1

Available Wards:
1. General Ward (LKR 3000.00/day)
2. Paediatric Ward (LKR 6000.00/day)
3. Surgical Ward (LKR 12000.00/day)
4. ICU (Intensive Care Unit) (LKR 25000.00/day)
Select Ward ID (1-4): 3
Days Admitted: 5
Assigned to Surgical Ward, Bed #01

Patient registered successfully. Patient ID: PAT-1001

=====
SMART HOSPITAL ADMISSION & BILL
=====
Patient ID      : PAT-1001
Patient Name    : N.P Perera
Age             : 65 Years
Specialty       : Neurology
Assigned Ward   : Surgical Ward (Bed #01)
Urgency Level   : Level 3 (Critical)
=====
Base Consultation Fee : LKR 5,000.00
Emergency Surcharge   : LKR 2,500.00 (50%)
Ward Stay Cost (5 Days) : LKR 60,000.00
=====
Gross Total Bill      : LKR 67,500.00
Age Subsidy Discount  : LKR -0.00 (0%)
=====
Final Payable Amount  : LKR 67,500.00
Estimated Waiting Time : 0.00 mins (Immediate Attention)
=====

=====
SMART HOSPITAL MENU
=====
1. Register New Patient (auto-generates bill)
2. View Bed Occupancy Status
3. View Patients in Priority Order
4. View Summary Report

```

6. Testing

Test Case	Input	Expected Result	Actual Result
Age exactly 5	age = 5	No discount (boundary excluded)	Pass
Age exactly 65	age = 65	No discount (boundary excluded)	Pass
Age 4	age = 4	15% discount applied	Pass
Age 66	age = 66	15% discount applied	Pass
Critical patient	urgency = 3	Wait time shown as 0.00 (Immediate Attention)	Pass
Ward fully occupied	Fill all 5 ICU beds, admit a 6th	Program rejects and asks for a different ward	Pass
Invalid urgency	urgency = 5	Re-prompts until 1-3 entered	Pass
Invalid menu choice	choice = 9	"Invalid choice" message, menu re-shown	Pass
Bed status persistence	Register + admit, exit, relaunch	ICU shows same occupied bed from beds status.txt	Pass

7. GitHub Repository and Assumptions

Repository: <https://github.com/sulanipomodya/smart-hospital-system>

Assumptions made during development:

- Estimated waiting time uses the specialty's queue count *after* the current patient has joined it, since specialtyQueueCount is incremented as soon as a specialty is selected during registration.
- Critical (Level 3) patients are always shown receiving immediate attention (0.00 min wait) on their bill, since triage means they're seen ahead of the normal queue regardless of the formula's raw output for their specialty.
- The Daily Patient Cap in the specialty lookup table is stored for reference but not enforced as a hard registration limit, since the brief doesn't explicitly require blocking new patients once a cap is reached.
- Age discount boundaries are strict: exactly 5 or exactly 65 does NOT qualify, matching the formula (age < 5 OR age > 65) as written.
- Bed assignment picks the first available free bed slot (lowest index) in the chosen ward.
- Invalid input re-prompts the user rather than crashing or exiting.

8. Challenges Faced and Future Improvements

Some genuine challenges from building this (rewrite based on what actually gave you trouble): keeping nine parallel arrays in sync for every patient without accidentally forgetting to update one of them; implementing comma thousand-separator currency formatting by hand since C's `printf` doesn't support it natively; deciding exactly when the specialty queue count should increment relative to calculating that same patient's wait time; and setting up a proper `Code::Blocks` project (vs. opening loose files) so the multi-file structure would actually compile.

With more time, the system could be extended to enforce each specialty's daily patient cap, support editing or discharging an already-registered patient, or migrate from parallel arrays to a struct `Patient` for cleaner code, as the assignment brief allows as an optional stretch goal.